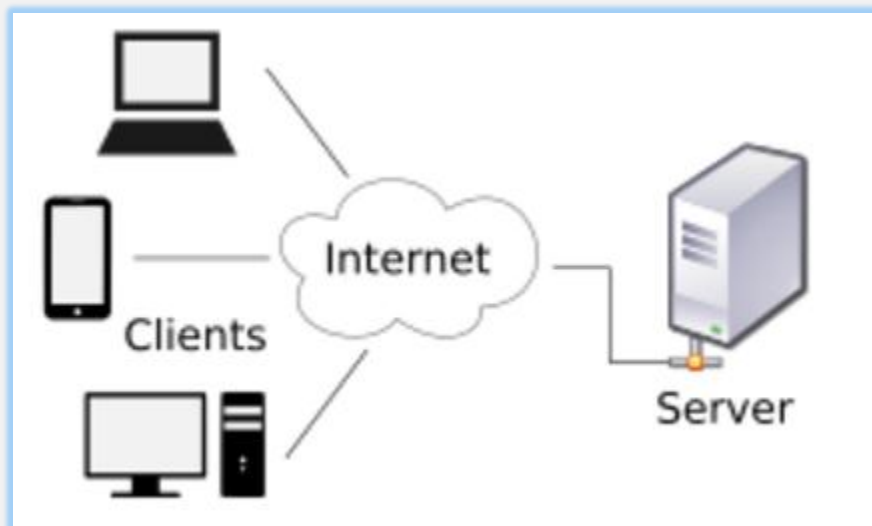


Building Blocks of Web-based Applications

Client-Server Architecture

- ❖ Client-server architecture in web development is a distributed computing model where functionality is split between two main components: clients (typically web browsers) and servers (web servers and application servers).
- ❖ The client is usually a web browser running on a user's device. It's responsible for presenting the user interface, handling user interactions, and making requests to servers.
- ❖ The server hosts the web application's core business logic, manages databases, handles authentication, and processes client requests.
- ❖ Communication happens primarily through HTTP/HTTPS protocols. Clients send requests (GET, POST, PUT, DELETE) to specific server endpoints, and servers respond with data, typically in JSON format for APIs or HTML for traditional web pages. Modern applications often use RESTful APIs.



https://en.wikipedia.org/wiki/Client%E2%80%93server_model

REST

(What is REST, 2020)

- ❖ It is an acronym: **RE**presentational **S**tate **T**ransfer
- ❖ An architectural style for distributed hypermedia systems
 - First presented in Roy Fielding's dissertation back in 2000

Resource

- ❖ A named information element
 - E.g.,
 - A document or image

- A temporal service
 - A collection of other services
 - An object (e.g., a person)
 - Etc.
- ❖ Resource representation
- The state of the resource on the server at any moment in time, which consists of:
 - The data
 - The metadata to describe the data
 - The hyperlinks that help the clients transition to the next desired state

6 Guiding Constraints

1. Uniform interface

1.1. Four interface constraints to achieve a uniform REST interface

- 1.1.1. **Identification of resources** – Every resource (data or functionality) is identified by a unique URI (like `/api/users/123`)
- 1.1.2. **Manipulation of resources through representations** – the resources should have uniform representations in the server response. API consumers should use these representations to modify the resources state in the server
 - 1.1.2.1. Common methods
 - HTTP GET to query a resource
 - HTTP PUT to update a resource
 - HTTP POST to add a resource
 - HTTP DELETE to delete a resource
- 1.1.3. **Self-descriptive messages** – each resource representation should carry enough information to describe how to process the message. The client processes the resource based on its self-described media type.
 - See: <https://www.iana.org/assignments/media-types/media-types.xhtml>
 - E.g., json, xml
- 1.1.4. **Hypermedia as the engine of application state** – The client should dynamically derive all other resources and interactions based on the initial URI of the application using hyperlinks.

Example: A GET request to `/books/456` returns book details along with links to edit, delete, or view related books.

2. Client-Server to enforce separation of concerns

This constraint separates user interface concerns from data storage concerns. The client and server are independent systems that communicate only through a standardized interface.

- Benefits: Each side can evolve independently, improving scalability and portability
- Client responsibilities: User interface, user experience, gathering user input

- Server responsibilities: Data storage, business logic, security, computing resources

Example: A mobile app (client) can work with the same server as a web browser, without the server needing to know about different client implementations.

3. **Stateless**

The server doesn't store any client context between requests. Each request from the client must contain all the information needed to understand and process that request.

- No session storage on server: All session information is kept on the client side
- Each request is independent: Every request contains authentication tokens, user preferences, etc.
- Benefits: Improves reliability, visibility, and scalability of the system

Example: Instead of storing "User is logged in" on the server, each request includes an authentication token proving the user's identity.

4. **Cacheable**

Responses from the server can be cached to improve performance and reduce network traffic.

- Cache headers: Servers include information about whether responses can be cached and for how long
- Types of caching: Browser cache, proxy cache, CDN cache
- Benefits: Faster response times, reduced server load, better user experience

Example: A product image might be cached for 24 hours, so repeated requests for the same image don't require hitting the server again.

5. **Layered system**

The REST architecture can have multiple layers between client and server, with each layer only knowing about adjacent layers.

- Examples of layers: Load balancers, proxy servers, gateways, firewalls
- Transparency: Clients don't need to know about intermediate layers
- Benefits: Improved scalability, security, and flexibility

Example: A client request might go through a load balancer, then a caching proxy, before reaching the application server, but the client only sees the final response.

6. **[Optional] Code on demand**

The server can send executable code to the client, extending client functionality dynamically.

- Optional constraint: This is the only constraint that's not required

- Examples: JavaScript sent from a web server, applets, plugins
- Benefits: Simplifies client development, allows dynamic updates
- Drawbacks: Reduces visibility and can create security concerns

Example: A web application sends JavaScript code that creates dynamic forms or interactive charts that run in the browser.

URL

- ❖ The **Uniform Resource Locator** is used to address accessible resources and services on the web. It is a subset of URIs.
 - A URI is a URL when it identifies the resource, and it provides a means of locating that resource.
 - **Web Resource** – a document or service made available through the WWW.
- ❖ Dafydd Stuttard, in The Web Application Hacker's Handbook he states: *"The correct technical term for a URL is actually URI (or uniform resource identifier), but this term is really only used in formal specifications and by those who wish to exhibit their pedantry."*

General notation:

`scheme://host[:port]/path/[/;url-params][?query-string][#anchor]`

Example

`http://www.website.com/sp/test?id=1234?num=2345&valid=true#middle`

Elements

- ❖ **Scheme:** the underlying protocol (e.g., `http`)
- ❖ **Host:** The IP address of the web server (e.g., `www.website.com`)
- ❖ **Port:** The port number at which the web server listens, default = 80
- ❖ **Path:** Logically, the path through the file system from the "root" directory, although the web server may interpret this in different ways (e.g., `/sp/test`). The path describes how to find the requested resource on the host machine.
- ❖ **URL-params:** Name-value pairs – can be used for session identifiers (e.g., `;id=1234`)
- ❖ **Query-string:** Name-value pairs – represents data being transferred with the request (e.g., `?num=2345&valid=true`)
- ❖ **Anchor:** A positional marker within the requested document (E.g., `#middle`)

URI examples

- ❖ `ftp://ftp.is.co.za/rfc/rfc1808.txt`
- ❖ `http://www.ietf.org/rfc/rfc2396.txt`
- ❖ `ldap://[2001:db8::7]/c=GB?objectClass=one`
- ❖ `mailto:John.Doe@example.com`
- ❖ `news:comp.infosystems.www.servers.unix`
- ❖ `tel:+1-816-555-1212`
- ❖ `telnet://192.0.2.16:80/`
- ❖ `urn:oasis:names:specification:docbook:dtd:xml:4.1.2`

HTTP (Hypertext Transfer Protocol)

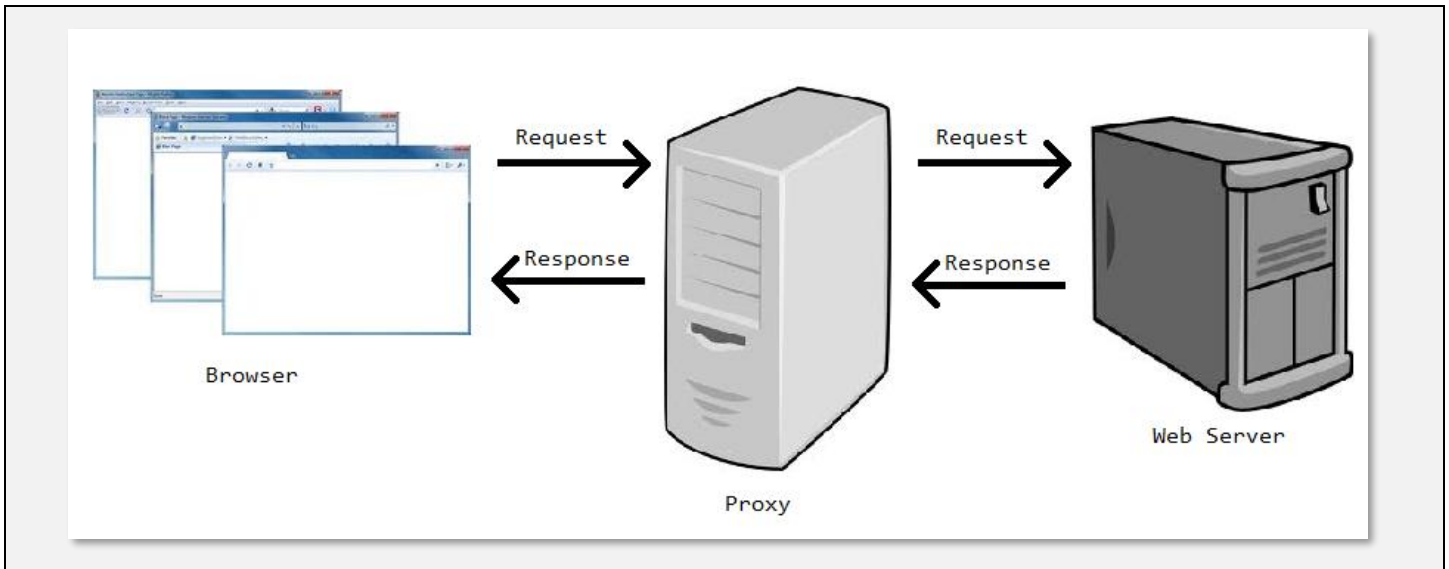
- ❖ The protocol is used to transport messages.
- ❖ It uses a request-response paradigm and is a **stateless** protocol.
 - Stateless means the HTTP server does not retain information over the duration of multiple requests.
 - **Cookies** allow the use of **stateful** sessions.
 - A cookie is information that a website stores on a user's computer.
- ❖ HTTP/1.1 is the version that is in common use, although HTTP/2 was standardized in 2015 and is supported by major web servers. HTTP/2 is not human-readable, and messages are encapsulated in frames. HTTP/3 is currently in Internet Draft as of January 2022.
- ❖ Common HTTP features:
 - **Caching** – allows the reuse of previously fetched resources. It helps to make web sites more responsive.
 - **Relaxing the origin constraint** – Only pages from the same origin can access all the information of a web page. HTTP headers can be used to relax this constraint.
 - **Authentication** – Basic authentication may be provided by using certain headers or by setting specific sessions using cookies
 - **Proxy and tunneling** – Proxy servers and HTTP tunnels facilitate access to WWW content. Forward proxies store and forward Internet services (e.g., DNS) to reduce and control bandwidth. Reverse proxies act on behalf of the servers and can be used for: load balancing, caching static content, and compression. Tunneling transmits private network data and protocol information through public network by encapsulating the data. A common use case for HTTP tunneling is HTTPS connections. Since the proxy can't decrypt HTTPS traffic, tunneling allows encrypted data to pass through transparently.

HTTP Flow Server-Side

1. Create and bind to socket, listen for client connections
2. Accept incoming client connection
3. Read HTTP request message from client
4. Process the request and generate response
5. Send HTTP response to client
6. Either close connection or keep it open for additional requests based on Connection header

HTTP Flow Client-Side

1. Client establishes TCP connection to server: open new connection, reuse existing connection, or open several connections
2. Send an HTTP message
3. Read the response sent by the server
4. Close or reuse the connection for further requests



HTTPS

- ❖ Hypertext Transfer Protocol Secure
- ❖ Specified by [RFC 2818](#)
- ❖ Uses port 443 by default
- ❖ Adds encryption, authentication, and integrity to the HTTP

SSL and TLS

- ❖ SSL
 - **Secure Sockets Layer**
 - Developed by Netscape
 - 2014 POODLE attack essentially killed SSL
- ❖ TLS
 - **Transport Layer Security**
 - Defined in 1999 as an upgrade to SSL 3.0
 - Current standard
 - Nowadays when people say SSL, they mean TLS

Generic HTTP Message Format

```

start-line
header-field-1
header-field-2
...
header-field-n
empty-line
[message-body]
  
```

Key components:

- Start line differs for requests vs responses:
 - Request: Method <space> Request-URI <space> HTTP-Version <CRLF>
 - Response: HTTP-Version <space> Status-Code <space> Reason-Phrase CRLF
- Headers: Name: Value pairs

- Empty line: Marks end of headers
- Message body: Optional payload (length specified by Content-Length header)

HTTP Request

- ❖ Is made when you:
 - Type a URL in the browser and press enter.
 - Follow a link.
 - Submit a form.
- ❖ The web client creates the HTTP request.

Example GET Request

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: en-US
Accept-Encoding: gzip, deflate
Connection: keep-alive
```

This shows:

- Start line with method (GET), URI (/index.html), and HTTP version
- Common headers specifying host, client details, accepted formats
- Empty line marking header end
- No message body (typical for GET requests)

Example POST Request

```
POST /submit-form HTTP/1.1
Host: api.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 57
Accept: */*
User-Agent: Mozilla/5.0
Connection: keep-alive

firstName=John&lastName=Doe&email=john.doe%40example.com&age=25
```

HTTP Response

- ❖ It is made in response to a request.

Example Response

```
HTTP/1.1 200 OK
Date: Mon, 23 Dec 2024 12:00:00 GMT
Server: Apache/2.4.52
Content-Type: text/html
Content-Length: 138
Last-Modified: Mon, 23 Dec 2024 11:55:00 GMT
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
```

```
Connection: close
```

```
<html>
<head>
  <title>Example Page</title>
</head>
<body>
  <h1>Hello World!</h1>
  <p>Welcome to this example page.</p>
</body>
</html>
```

HTML (Hypertext Markup Language)

- ❖ HTML is the language used to markup web documents to derive semantic intent (meaning)
- ❖ HTML's original purpose was to enable cross-referencing of documents through hyper-links.
- ❖ The document is described by HTML tags.
 - HTML tags are keywords surrounded by angle brackets:

```
<tagname>...content...</tagname>
```

E.g., `<p>This is a paragraph</p>`

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>...</title>
</head>
<body>
  <header>...</header>
  <nav>...</nav>
  <article>
    <section>
      ...
    </section>
  </article>
  <aside>...</aside>
  <footer>...</footer>
</body>
</html>
```

- ❖ How send a HTTP GET request using the `<a>` element

```
<a href="url">link text</a>
```

For example,

```
<a href="https://www.etsu.edu/ehome/">ETSU</a>
```

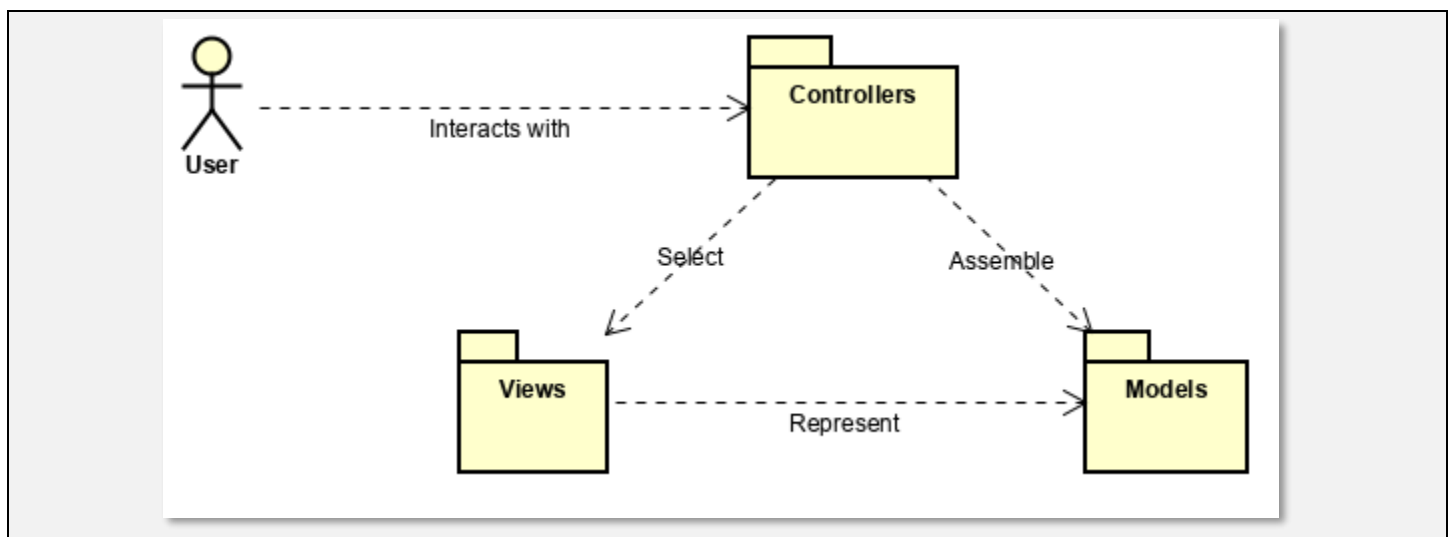
- ❖ How to send an HTTP GET request using the `<form>` element
- ❖ The data is appended to the URL in key value-pairs

```
<form action="/some_url" method="get">
  <!-- Other inputs as needed -->
  <input type="submit" value="Submit">
</form>
```

- ❖ How to send an HTTP POST request using the <form> element
- ❖ The data is included in the body of the request

```
<form action="/some_url" method="post">
  <!-- Other inputs as needed -->
  <input type="submit" value="Submit">
</form>
```

Model View Controller (MVC)



- ❖ An architectural design pattern that focuses on the separation of presentation from the data domain
 - It improves the maintainability and testability of the code

Model

The model is the data domain. It stores the data elements.

Example

Student Data

First Name: Betty
Last Name: Rubble
Date of Birth: 1/1/1900
Gender Identity: Female
Sex: Female

View

- ❖ The view is the visual representation of the model.

❖ For example – a data entry view and a student details view

The image displays two screenshots of a web application interface, both titled 'Student Data' in the browser window header. The interface includes a navigation bar with back, forward, and refresh buttons, and a URL bar showing 'https://'. The main content area contains a form with the following fields:

- Enter new student data:**
 - First Name:
 - Last Name:
 - Date of birth: (with a calendar icon)
 - Gender Identity: (with a dropdown arrow)
 - Sex: (with a dropdown arrow)
 - Buttons: and
- Student Details:**
 - First Name:
 - Last Name:
 - Date of birth: (with a calendar icon)
 - Gender Identity: (with a dropdown arrow)
 - Sex: (with a dropdown arrow)
 - Buttons: and

Controller

- ❖ The controller represents the interface between the user of the system and the system itself (note: not the user interface)
- ❖ It builds the model, tells the model when to update, and selects the appropriate views depending on commands issued by the user.

Advantages of MVC

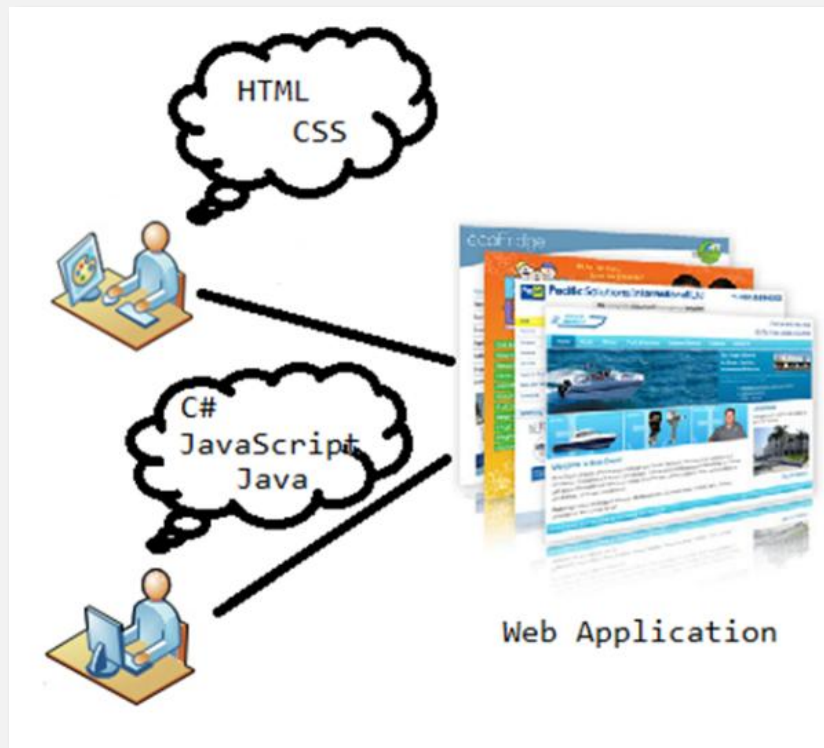
- ❖ It makes managing complexity easier.
- ❖ It improves the testability of your application.
- ❖ It works for large teams of developers and for Web designers who need a high degree of control over the application behavior.

Disadvantages of the MVC

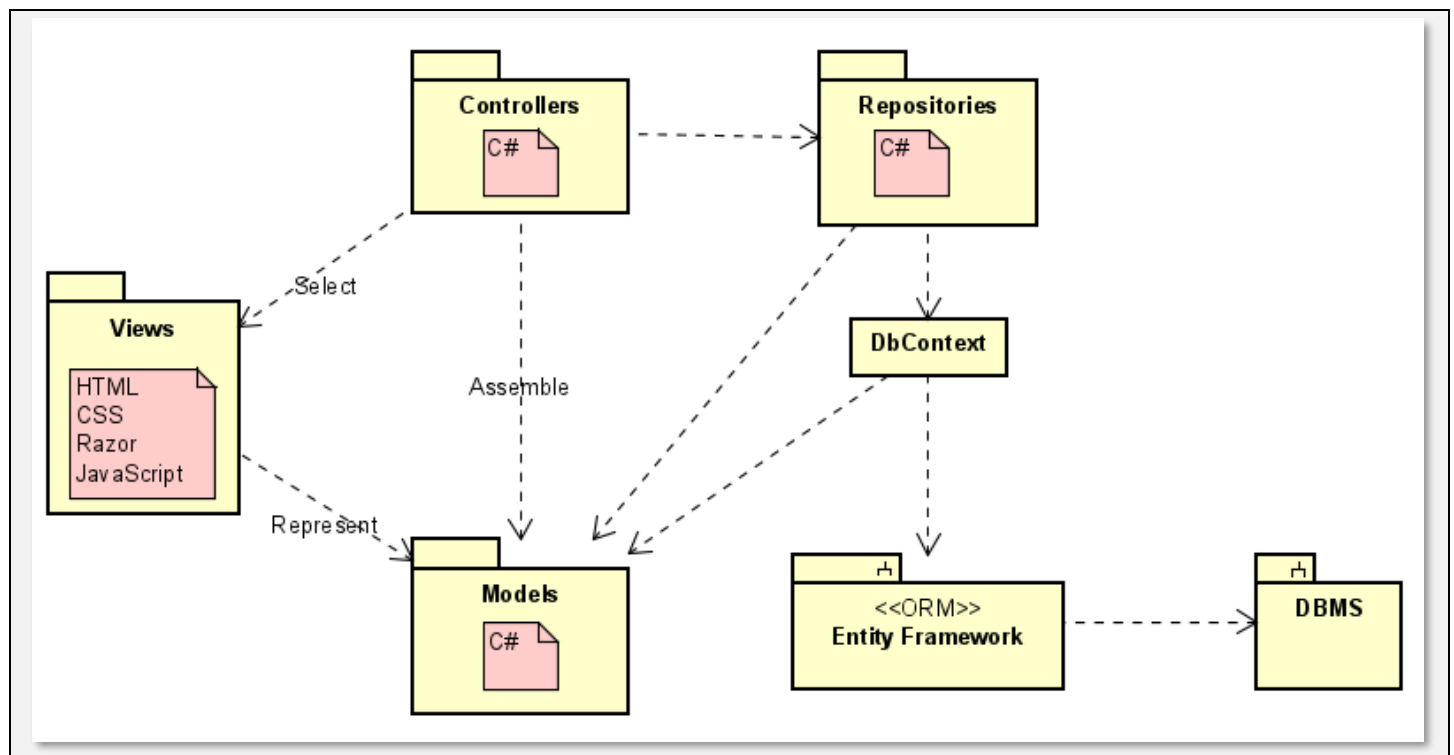
- ❖ Some degree of increased complexity
- ❖ More complex file structure
- ❖ Can be challenging to maintain the layers

Software Framework

- ❖ Typically includes support programs, compilers, code libraries, tool sets, and application programming interfaces (APIs) that bring together all the different components to enable development of a project or system.
- ❖ Using a framework allows the design principle of separating content from presentation.
 - Logic and formatting are not encapsulated in a single module.
- ❖ Frameworks provide a consistent infrastructure that reduces the need to write common application code.
- ❖ The best frameworks allow developers and designers to work on separate modules with little interference.



Software Architecture for this Course



- ❖ Repositories will be used to interact with the database through the database context class which uses the Entity Framework ORM

Object-Relational Mapping (ORM)

- ❖ Definition: Technology integrating object programming with relational databases
- ❖ Purpose: Makes database objects appear as programming language objects
- ❖ Benefits:
 - Eliminates most data-access code
 - Provides consistent interface
 - Simplifies database operations
- ❖ Popular ORM examples:
 - Java: Hibernate
 - .NET: Entity Framework
 - PHP: Laravel
 - Python: Django

Entity Framework

- ❖ Definition: Object-relational mapper for .NET developers
- ❖ Purpose: Work with relational data using domain-specific objects
- ❖ Key benefit: Eliminates need for most data-access code
- ❖ Features:
 - Code-first approach
 - Database-first approach
 - LINQ query support
 - Change tracking
 - Lazy loading
- ❖ Integration: Works seamlessly with repository pattern in our architecture